DEVELOPER CODING PLAN

Online RFQ & Tendering Platform

Step-by-Step Guide for Novice Developers

Based on Technical PRD v3.0 | Schema v3.0 | February 2026

6 Phases • 28 Tasks • Every step explained

Table of Contents

1. How to Read This Document

This plan is written assuming you know basic HTML, CSS, and a little JavaScript — but you have never built a full backend application before. Every single step is broken down so you never have to guess what to do next.

The project is divided into 6 Phases. Each Phase contains multiple Tasks. Each Task is one focused piece of work. Do them in the exact order shown. Do not skip ahead.

The Golden Rules

- Do one Task at a time. Finish it completely before moving to the next.
- After every Task, run the app and confirm it still works.
- If something breaks, stop and fix it before continuing.
- Copy-paste commands exactly as written. Typos in terminal commands cause confusing errors.
- Google is your friend — if a concept is unclear, search for "[concept] explained simply".

2. Phase Overview Map

This table shows you the big picture. Each phase builds on the one before it. The "Unlocks" column tells you what becomes possible once you finish that phase.

Phase	Name	What You Build	Unlocks
1	Project Setup & Foundation	Folder structure, database, basic server, login page	Everything else — this is the foundation.
2	Tender & Line-Item Core	Create/publish tenders, add line items and features	Vendors can see tenders. Bidding becomes possible.
3	Vendor & Bid Engine	Vendor registration, bid submission with versioning & envelopes	The core buy-sell transaction works end to end.
4	Evaluation & Awards	Two-stage scoring, comparison matrix, award issuance	A tender can be completed from start to finish.
5	Notifications & Addenda	Email/in-app alerts, Q&A forum, official amendments	The platform feels complete to real users.
6	Export, Tax & Polish	PDF/XLSX exports, tax engine, FX rates, audit logs	Production-ready. All PRD features shipped.

3. Before You Write Any Code — Environment Checklist

Install every item below before you start Phase 1. These are the tools the entire project depends on. You install them once and they stay installed.

#	Tool	What It Does	Install Command (run in Terminal)
1	Node.js (v20 LTS)	Runs your backend server and frontend build	Download from https://nodejs.org — pick "LTS"
2	npm	Installs JavaScript packages (comes with Node.js)	Already included with Node.js — no action needed
3	PostgreSQL 16	Your database — where all data is stored	Download from https://www.postgresql.org/download
4	pgAdmin 4	A visual tool to look at and manage your database	Download from https://www.pgadmin.org
5	Redis	A fast in-memory store for sessions and caching	Download from https://redis.io/download
6	Git	Tracks changes to your code (version control)	Download from https://git-scm.com/download
7	VS Code	Your code editor — where you write everything	Download from https://code.visualstudio.com
8	Postman	A tool to test your API endpoints manually	Download from https://www.getpostman.com

⚠ **Important:** After installing PostgreSQL, remember the password you set for the 'postgres' user. You will need it later. Write it down somewhere safe.

4. Phase 1: Project Setup & Foundation

This phase creates the skeleton of the entire application. By the end, you will have a working server that can show a login page and talk to a database. Nothing fancy yet — but everything later is built on top of this.

Task 1 — Create the Project Folder Structure

This sets up the exact folder layout every file will live in. Following this structure keeps things organized as the project grows.

```
mkdir rfq-platform && cd rfq-platform
mkdir -p backend/src/{routes,middleware,controllers,models,services,utils}
mkdir -p backend/src/config
mkdir -p frontend/src/{pages,components,styles,utils}
mkdir -p frontend/public
```

Your folder should now look like this:

```
rfq-platform/
  backend/
    src/
      config/        ← database config, env variables
      controllers/   ← handle HTTP requests
      middleware/    ← auth checks, error handling
      models/        ← database table definitions
      routes/        ← URL path definitions
      services/      ← business logic
      utils/         ← helper functions
  frontend/
    src/
      pages/         ← each screen/page of the app
      components/    ← reusable UI pieces
      styles/        ← CSS files
      utils/         ← frontend helper functions
    public/          ← images, icons, static files
```

💡 **Note:** Think of controllers as the "front door" that receives requests. Services do the actual thinking. Models talk to the database.

Task 2 — Initialize Backend with package.json

This tells Node.js what your project is and what packages it needs.

```
cd rfq-platform/backend
npm init -y
```

Then install the core packages your backend needs:

```
npm install express pg uuid bcryptjs jsonwebtoken dotenv cors helmet
```

Package	Why You Need It
express	The web framework — handles HTTP requests and routes.
pg	PostgreSQL driver — lets Node.js talk to your database.
uuid	Generates unique IDs (UUID v4) for database rows.
bcryptjs	Hashes passwords securely before storing them.
jsonwebtoken	Creates and verifies JWT tokens for login sessions.
dotenv	Reads secret values (like passwords) from a .env file.
cors	Allows your frontend to talk to your backend.
helmet	Adds security headers to every response automatically.

Task 3 — Create the .env Configuration File

This file holds secret values that should never be committed to Git. Create it in the backend/ folder.

```
# backend/.env
DB_HOST=localhost
DB_PORT=5432
DB_NAME=rfq_platform
DB_USER=postgres
DB_PASSWORD=your_postgres_password_here
JWT_SECRET=write_a_random_string_here_at_least_32_chars
PORT=3000
```

⚠ **Important:** Replace 'your_postgres_password_here' with the actual password you set when installing PostgreSQL. Replace the JWT_SECRET with any random string of at least 32 characters.

Now create a .gitignore so these secrets are never saved to Git:

```
# backend/.gitignore
node_modules/
.env
```

Task 4 — Set Up the Database

Open pgAdmin, connect to your PostgreSQL server, and create a new database called rfq_platform.

1. Open pgAdmin 4 and connect to your local PostgreSQL server using the password from your .env file.
2. Right-click on "Databases" in the left panel → click "Create" → "Database…"
3. Type rfq_platform as the database name. Click Save.
4. Now open the Query Tool: click on rfq_platform → then click the lightning-bolt icon (Query Tool) at the top.
5. Open the schema file (rfq_tendering_platform_schema_v3.sql) in a text editor. Select all the text (Ctrl+A), copy it (Ctrl+C).
6. Paste it into the pgAdmin Query Tool (Ctrl+V). Click the Play button (▶) to run it.
7. If you see "Query returned successfully" — you are done. All 38 tables are now created.

💡 **Note:** If you see an error, read the error message carefully. It usually tells you exactly which line failed. The most common issue is a typo — double-check that line.

Task 5 — Create the Database Connection Module

This is the single file that connects your Node.js code to PostgreSQL. Every other file that needs the database will import this one.

Create the file backend/src/config/database.js with this content:

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host:     process.env.DB_HOST,
  port:     parseInt(process.env.DB_PORT),
  database: process.env.DB_NAME,
  user:     process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

module.exports = pool;
```

💡 **Note:** Pool keeps a set of connections open to the database so you don't have to open a new one every time. This is much faster.

Task 6 — Build the Main Server Entry Point

This is the file that starts your entire backend server. Create backend/src/app.js:

```
const express    = require('express');
const helmet     = require('helmet');
const cors       = require('cors');
require('dotenv').config();

const app = express();
app.use(helmet());
app.use(cors({ origin: 'http://localhost:3001' }));
app.use(express.json());

// Health check — confirms the server is alive
app.get('/health', (req, res) => res.json({ status: 'ok' }));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

module.exports = app;
```

Test it: run node src/app.js from the backend/ folder. Open your browser and go to http://localhost:3000/health. You should see {"status":"ok"}.

Task 7 — Implement User Registration & Login

This creates the auth system. Users can sign up, log in, and receive a JWT token that proves who they are for all future requests.

Files to create:

- backend/src/controllers/authController.js — handles the register and login HTTP requests
- backend/src/routes/authRoutes.js — defines POST /api/auth/register and POST /api/auth/login
- backend/src/middleware/authMiddleware.js — a function that checks the JWT token on protected routes

authController.js — core logic:

```
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');
const { v4: uuidv4 } = require('uuid');
const db     = require('../config/database');

// REGISTER
exports.register = async (req, res) => {
  try {
    const { name, email, password, organizationId, roles } = req.body;
    const hash = await bcrypt.hash(password, 12);  // cost factor 12
    const id   = uuidv4();
    await db.query(
      `INSERT INTO users (id, organization_id, name, email, password_hash, roles)
       VALUES ($1,$2,$3,$4,$5,$6)`,
      [id, organizationId, name, email, hash, roles || ['vendor']]
    );
    res.status(201).json({ message: 'User created', id });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// LOGIN
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const { rows } = await db.query('SELECT * FROM users WHERE email = $1', [email]);
    if (!rows[0]) return res.status(401).json({ error: 'Invalid credentials' });
    const match = await bcrypt.compare(password, rows[0].password_hash);
    if (!match) return res.status(401).json({ error: 'Invalid credentials' });
    const token = jwt.sign(
```

```
      { id: rows[0].id, roles: rows[0].roles, orgId: rows[0].organization_id },
      process.env.JWT_SECRET,
      { expiresIn: '15m' }
    );
    res.json({ token });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

authMiddleware.js — protects routes:

```
const jwt = require('jsonwebtoken');

exports.authenticate = (req, res, next) => {
  const header = req.headers.authorization;
  if (!header) return res.status(401).json({ error: 'No token' });
  const token = header.split(' ')[1];  // 'Bearer <token>'
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();  // token is valid — continue to the route handler
  } catch (e) {
    res.status(401).json({ error: 'Invalid token' });
  }
};
```

Wire it into app.js by adding these two lines before the health check:

```
const authRoutes = require('./routes/authRoutes');
app.use('/api/auth', authRoutes);
```

Test: Use Postman to POST to http://localhost:3000/api/auth/register with a JSON body containing name, email, password, and organizationId. Then POST to /api/auth/login with email and password. You should get back a token.

5. Phase 2: Tender & Line-Item Core

Now the database talks to your server and users can log in. This phase builds the heart of the platform: creating tenders, adding line items, attaching features, and publishing tenders so vendors can see them.

Task 8 — Build the Tender CRUD API

CRUD stands for Create, Read, Update, Delete — the four basic operations on any piece of data.

Create backend/src/controllers/tenderController.js with these endpoints:

Method	URL	What It Does	Who Can Call It
POST	/api/tenders	Creates a new tender in draft status	Buyer, Admin
GET	/api/tenders	Lists all tenders (filtered by role/status)	Any logged-in user
GET	/api/tenders/:id	Gets one tender by its ID	Any logged-in user (access rules apply)
PUT	/api/tenders/:id	Updates a tender (only while in draft)	Buyer (owner), Admin
DELETE	/api/tenders/:id	Soft-cancels a tender (sets status = cancelled)	Buyer (owner), Admin

Key implementation details:

- On POST: auto-generate tender_number using a pattern like RFQ-YYYY-XXXXX (year + sequential number). Set status = 'draft' and created_by = the logged-in user's ID.
- On PUT: first check that the current status is 'draft'. If it isn't, return HTTP 409 with message "Cannot edit — tender is not in draft status".
- On GET (list): if the user is a Vendor, only return tenders where visibility = 'open' OR where they have a row in tender_vendor_invitations.
- Always insert a row into audit_logs after any successful create, update, or delete.

Task 9 — Implement Tender Status Transitions

A tender moves through a strict sequence of statuses. This task builds the function that enforces those rules. Create it in backend/src/services/tenderService.js.

From	→ To	The API call that triggers this
draft	published	PUT /api/tenders/:id/publish
draft	cancelled	PUT /api/tenders/:id/cancel
published	clarification	Automatic — when a vendor posts a question
published	closed	Automatic — when submission_deadline passes
clarification	published	Automatic — when buyer answers the question

closed	tech_eval	PUT /api/tenders/:id/open-bids (manual) or automatic at bid_opening_time
tech_eval	comm_eval	PUT /api/tenders/:id/unlock-commercial
comm_eval	awarded	PUT /api/tenders/:id/award

Build a single function called transitionStatus(tenderId, newStatus) that:

- Looks up the current status from the database
- Checks if the transition is legal using a map of allowed moves
- If illegal: throws an error with HTTP 409
- If legal: updates the status and the updated_at timestamp
- Logs the transition in audit_logs with metadata: { old_status, new_status }

Task 10 — Build the Line-Item (BoM) API

Line items are the individual products or services inside a tender. They can be nested (a group contains sub-items) to form a Bill of Materials.

Method	URL	What It Does
POST	/api/tenders/:tenderId/items	Adds a new line item (or group) to the tender
GET	/api/tenders/:tenderId/items	Returns all items as a nested tree
PUT	/api/tenders/:tenderId/items/:id	Updates an item's details
DELETE	/api/tenders/:tenderId/items/:id	Removes an item (and all its children)

Critical rules to enforce:

- If item_type = 'group', quantity must be 0.
- If item_type = 'item', quantity must be > 0.
- parent_item_id must either be NULL (root item) or point to an existing item in the same tender.
- Items can only be added or changed while the tender is in 'draft' status.
- estimated_cost is never returned in any API response to a Vendor. Strip it out in the controller before sending.

💡 **Note:** To return items as a nested tree: first fetch all items for the tender flat from the DB, then use a simple JavaScript function to group children under their parents by matching parent_item_id.

Task 11 — Build the Feature Definition & Options API

Features are the structured questions a buyer defines per line item. For example: "Processor" with options ["Intel i5", "Intel i7", "AMD Ryzen 5"].

Method	URL	What It Does
POST	/api/features	Creates a new feature definition

POST	/api/features/:id/options	Adds an option to a feature
POST	/api/tenders/:tenderId/items/:itemId/features	Attaches a feature to a line item (mandatory or optional)
GET	/api/tenders/:tenderId/items/:itemId/features	Returns all features + options attached to a line item

Validation rules:

- evaluation_weight across all features on a single line item should sum to 100. Return a warning (not error) if it doesn't.
- A feature with is_global = true can be reused on any tender without re-creating it.
- feature_type determines what kind of input the vendor sees: single_select shows a dropdown, multi_select shows checkboxes, text shows a text box, numeric shows a number input, boolean shows a toggle.

Task 12 — Build the Publish Flow & Vendor Visibility

When a buyer publishes a tender, visibility rules kick in. This task wires up the two tender modes.

For Open tenders:

- On publish: set status = 'published'. No further action needed — all approved vendors can see it.

For Limited tenders:

- On publish: for each vendor the buyer invited, insert a row into tender_vendor_invitations with a random token.
- Generate the token like this: require('crypto').randomBytes(32).toString('hex')
- Set status = 'sent' on each invitation row.
- Queue a notification (Phase 5 will send the actual email — for now just insert a row into the notifications table with status = 'pending').

Vendor GET /api/tenders access rule:

- If tender.visibility = 'open' → always visible to approved vendors.
- If tender.visibility = 'limited' → only visible if the vendor has a row in tender_vendor_invitations for this tender.

6. Phase 3: Vendor & Bid Engine

Phase 2 built the buyer side. This phase builds the vendor side: registering as a vendor, submitting bids with proper versioning and envelope separation, and enforcing the encryption/hash rules from the PRD.

Task 13 — Build Vendor Profile & Enlistment API

Vendors must register a profile before they can bid on anything.

Method	URL	What It Does
POST	/api/vendors/profile	Creates a vendor profile (links to their organization)
POST	/api/vendors/profile/documents	Uploads a compliance document (trade license, VAT cert, etc.)
GET	/api/vendors/profile	Returns the current vendor's profile + documents
POST	/api/vendors/:orgId/approve	Buyer/Admin approves or rejects a vendor (body: { status })

Enlistment status rules (state machine):

- pending → approved or rejected
- approved → suspended
- rejected → pending (vendor can re-apply)
- suspended → approved or rejected
- Any other transition returns HTTP 409.

- When status is set to 'rejected', rejection_reason is mandatory in the request body.
- For document uploads: store the file in object storage (for now, just save the file_url). Track issued_date and expiry_date.

Task 14 — Build the Bid Submission API

This is the most complex single task. A bid has a header, line-item responses, and feature selections. It must be versioned and split into envelopes.

Method	URL	What It Does
POST	/api/tenders/:tenderId/bids	Creates a new bid (draft)
PUT	/api/tenders/:tenderId/bids/:bidId	Updates bid content (while in draft)
POST	/api/tenders/:tenderId/bids/:bidId/submit	Finalizes and submits the bid
POST	/api/tenders/:tenderId/bids/:bidId/withdraw	Withdraws the bid (before deadline)
GET	/api/tenders/:tenderId/bids/my	Returns the vendor's own active bid

Step-by-step what happens on /submit:

8. Check that the tender's submission_deadline has not passed. If it has, return HTTP 403 "Deadline passed".
9. Check that all mandatory features for every line item have a response in bid_item_feature_values. If any are missing, return HTTP 400 with a list of what's missing.
10. Check that all unacknowledged addenda are acknowledged by this vendor. If not, return HTTP 403 "Unacknowledged addenda".
11. Calculate item_total_price for each bid_item: quantity × unit_price.
12. Calculate total_amount: sum of all item_total_prices + taxes.
13. Compute digital_hash: SHA-256 of the entire bid payload (JSON.stringify the bid object). Store it in bids.digital_hash.
14. Set status = 'submitted' and submitted_at = now().
15. Create two rows in bid_envelopes: one with envelope_type = 'technical' (is_open = false), one with 'commercial' (is_open = false).
16. Insert a notification row for the vendor (confirmation) and the buyer.

Task 15 — Implement Bid Versioning Logic

Vendors can withdraw and resubmit. Each resubmission creates a new version. Only the latest non-withdrawn version counts.

On /withdraw:

- Set the current bid's status to 'withdrawn'. Do NOT delete it.
- The bid row stays in the database as history.

On next /submit (after a withdrawal):

- Look up the highest version number for this vendor + tender combination.
- Create a brand-new bid row with version = previous_max + 1.
- The old withdrawn row remains untouched.

Active bid query (use this everywhere you need "the current bid"):

```
SELECT * FROM bids
WHERE tender_id = $1 AND vendor_org_id = $2 AND status = 'submitted'
ORDER BY version DESC LIMIT 1;
```

💡 **Note:** If this query returns zero rows, the vendor has no active bid for this tender.

7. Phase 4: Evaluation & Awards

Bids are in. Now evaluators score them, the buyer compares them side by side, and awards are issued. This phase implements the two-stage evaluation workflow.

Task 16 — Build the Bid Opening & Envelope System

Bids are sealed until the buyer opens them. This task controls when and how envelopes are unlocked.

Method	URL	What It Does
POST	/api/tenders/:tenderId/open-bids	Opens all technical envelopes for all bids
POST	/api/tenders/:tenderId/bids/:bidId/unlock-commercial	Opens the commercial envelope for one specific bid

Rules:

- open-bids: transition tender status from 'closed' to 'tech_eval'. Set is_open = true and opened_at = now() on every technical envelope. Log in audit_logs.
- unlock-commercial: can only be called during 'comm_eval' status. The bid must be marked is_technically_qualified = true in evaluations. Set is_open = true on the commercial envelope.
- Before opening, verify the digital_hash: recompute SHA-256 of the bid payload and compare to bids.digital_hash. If mismatch → flag as TAMPERED, do not open, log the event.

Task 17 — Build the Evaluation & Scoring API

Evaluators score each bid's technical compliance feature by feature.

Method	URL	What It Does
POST	/api/tenders/:tenderId/bids/:bidId/evaluate	Submits a full evaluation for one bid
GET	/api/tenders/:tenderId/evaluations	Returns all evaluations for a tender

The POST body contains:

- technical_score: the evaluator's overall technical score (0–100)
- line_scores: an array of { tender_item_id, feature_id, score, remarks } — one entry per feature per item
- remarks: free-text notes on the overall evaluation

The backend calculates:

- Saves each line_score row into evaluation_line_scores
- Sets is_technically_qualified = true if technical_score >= the tender's configured minimum threshold (default: 70)
- After commercial unlock: commercial_score is calculated by the system based on price ranking

- overall_score = (tech_weight × technical_score) + (comm_weight × commercial_score). Default weights: 60% tech, 40% commercial

Task 18 — Build the Comparison Matrix API

This is the evaluator's main view — a side-by-side comparison of all bids.

Method	URL	Returns
GET	/api/tenders/:tenderId/comparison	Full comparison matrix: compliance + features + pricing
GET	/api/tenders/:tenderId/comparison/ranking	Ranked list of bids by overall_score

The comparison response structure:

- compliance_matrix: for each (vendor × line_item), return 'compliant', 'partial', or 'non_compliant'
- feature_comparison: for each line_item, show each vendor's selected option side by side
- financial_comparison: a grid where rows = line items, columns = vendors. Each cell has unit_price and item_total_price. Last row = grand total including taxes
- Only include commercial data if the vendor's commercial envelope is_open = true

Task 19 — Build the Award API

The buyer awards one or more line items to one or more vendors (split awards are allowed).

Method	URL	What It Does
POST	/api/tenders/:tenderId/awards	Issues an award for a specific line item to a specific bid
GET	/api/tenders/:tenderId/awards	Returns all awards for a tender

POST body: { tender_item_id, bid_id, awarded_quantity, awarded_price }

Validation rules:

- awarded_quantity must be ≤ the tender_item's original quantity.
- The sum of all awarded_quantity values across all awards for one tender_item must not exceed that item's quantity.
- The bid must belong to this tender.
- The bid must have is_technically_qualified = true.
- After all line items are awarded, transition the tender status to 'awarded'.

8. Phase 5: Notifications & Addenda

The platform can now complete a full tender lifecycle — but users have no way to know what's happening. This phase adds the communication layer: notifications, the Q&A forum, and official amendments.

Task 20 — Build the Notification Service

Every important event in the system triggers a notification. Earlier phases inserted rows into the notifications table with status = 'pending'. This task actually sends them.

Install the email package:

```
npm install nodemailer
```

Create backend/src/services/notificationService.js:

- A function sendPending() that queries all notifications WHERE status = 'pending'
- For each: check the channel (email, sms, in_app)
- For email: use Nodemailer to send. On success → status = 'sent'. On failure → status = 'retried' (up to 3 times with delays: 1 min, 5 min, 25 min). After 3 failures → status = 'failed'.
- For in_app: this will be delivered via WebSocket in a later enhancement. For now, just set status = 'sent' immediately.

Run sendPending() on a timer — check every 30 seconds for pending notifications.

💡 **Note:** For the email configuration: create a free account on SendGrid or use Gmail's SMTP. Put the SMTP credentials in your .env file.

Task 21 — Wire Up All Notification Triggers

Go back through your previous controllers and make sure every important action inserts a notification row. Here is the complete checklist:

Event	Who Gets Notified	Which Controller
Tender published	All eligible / invited vendors	tenderController → publish
Invitation sent	The invited vendor	tenderController → publish (limited)
Addendum published	All invited / eligible vendors	addendumController → create
Addendum not acknowledged (24h warning)	Vendors with pending acks	notificationService → scheduled check
Clarification answered	The asking vendor (+ all if public)	clarificationController → answer
Deadline reminder (3 days)	Vendors who haven't submitted	notificationService → scheduled check

Deadline reminder (1 day)	Vendors who haven't submitted	notificationService → scheduled check
Bid submitted	The vendor + the buyer	bidController → submit
Bids opened	Buyer + all evaluators	tenderController → open-bids
Tender awarded	All participating vendors + buyer	awardController → award
Vendor doc expiry (30 days)	Vendor contacts + tenant admin	notificationService → scheduled check
Vendor doc expiry (7 days)	Vendor contacts + tenant admin	notificationService → scheduled check

Task 22 — Build the Clarification (Q&A) API

Vendors can ask questions about a published tender. Buyers answer them.

Method	URL	What It Does
POST	/api/tenders/:tenderId/questions	Vendor posts a question
POST	/api/tenders/:tenderId/questions/:id/answer	Buyer answers a question
GET	/api/tenders/:tenderId/questions	Lists all questions (filtered by visibility)

Rules:

- Questions can only be posted when tender status is 'published' or 'clarification'.
- When a question is posted, if the tender is in 'published' status, transition it to 'clarification'.
- is_public = true → all vendors see both the question and the answer. is_public = false → only the buyer and the asking vendor see it.
- When the buyer answers, if creates_addendum = true in the response body, the buyer must also create an addendum (see Task 23).
- After all open questions are answered, the tender transitions back to 'published'.

Task 23 — Build the Addenda API

When an answer changes the scope of a tender, the buyer publishes an official amendment. All vendors must acknowledge it before they can submit a bid.

Method	URL	What It Does
POST	/api/tenders/:tenderId/addenda	Publishes a new addendum
POST	/api/tenders/:tenderId/addenda/:id/acknowledge	Vendor acknowledges an addendum
GET	/api/tenders/:tenderId/addenda	Lists all addenda + acknowledgement status

On POST (create addendum):

- Set addendum_number = max(current addendum_numbers for this tender) + 1.
- If extends_deadline_days > 0: update the tender's submission_deadline by adding that many days.
- Insert a row into addendum_acknowledgements for every invited/eligible vendor with acknowledged_at = NULL.
- Send a notification to all vendors.

On /acknowledge:

- Set acknowledged_at = now() and acknowledged_by = the logged-in vendor user's ID.

⚠ **Important:** Bid submission (Task 14) must check: SELECT COUNT(*) FROM addendum_acknowledgements WHERE addendum_id IN (SELECT id FROM addenda WHERE tender_id = ?) AND vendor_org_id = ? AND acknowledged_at IS NULL. If count > 0, reject the bid.

9. Phase 6: Export, Tax & Polish

The core platform works. This final phase adds the remaining PRD features: tax calculation, currency conversion, PDF/XLSX exports, the complete audit log, and rate limiting. After this phase, the platform is production-ready.

Task 24 — Build the Tax Engine

Tax is calculated per line item based on rules configured by an admin.

Method	URL	What It Does
POST	/api/admin/tax-rules	Admin creates a tax rule (e.g., GST 18%)
GET	/api/admin/tax-rules	Lists all active tax rules
PUT	/api/admin/tax-rules/:id	Updates or deactivates a tax rule

When a bid is submitted (Task 14), after calculating item_total_price for each line item:

- Look up all active tax_rules where applies_to matches the tender's procurement_type OR applies_to = 'all'.
- For each matching rule: tax_amount = item_total_price × (rate_percent / 100). Insert into bid_item_taxes.
- Add all tax_amounts to the bid's total_amount.

Task 25 — Build the Currency Rate Cache

All tenders are in one currency. This module fetches exchange rates daily so that cross-currency comparisons are possible in reporting.

Create a scheduled job in notificationService.js (or a separate schedulerService.js) that runs once per day:

- Calls a free FX API (e.g., https://api.exchangerate.host/latest?base=USD)
- For each rate returned: UPSERT into currency_rates (insert if not exists, update if exists) with fetched_at = now().

💡 **Note:** 'Upsert' means: if a row with this base+target already exists, update the rate. Otherwise insert a new row. In PostgreSQL this is done with INSERT … ON CONFLICT … DO UPDATE.

Task 26 — Build the Export Engine (PDF & XLSX)

Buyers and evaluators can download reports. PDF generation runs asynchronously so it doesn't block the server.

Install the needed packages:

```
npm install pdfkit xlsx
```

Export	Format	Endpoint
Tender Summary	PDF	GET /api/tenders/:id/export/summary
Bid Comparison Matrix	PDF / XLSX	GET /api/tenders/:id/export/comparison?format=pdf
Bid Integrity Report	PDF	GET /api/tenders/:id/export/integrity
Award Letter	PDF	GET /api/tenders/:id/export/award-letter/:vendorId
Full Data Dump	XLSX	GET /api/tenders/:id/export/dump (Admin only)

Implementation pattern for PDF exports:

17. The endpoint receives the request and validates permissions (is the user allowed to export this?).
18. It inserts a job record into a simple jobs table (or uses the notifications table with a special type).
19. A background worker picks up the job, generates the PDF using pdfkit, saves it to object storage (S3/MinIO), and sends a notification to the user with a download link.
20. The user clicks the link in their notification to download the file.

💡 **Note:** For XLSX: the xlsx package has a writeFile function. For the Data Dump, stream rows from the database directly into the XLSX writer to avoid loading thousands of rows into memory at once.

Task 27 — Implement Rate Limiting

Rate limiting prevents abuse and protects the server during peak traffic (e.g., bid submission rush at deadline).

Install the rate limiting package:

```
npm install express-rate-limit
```

Add these three rate limiters in app.js before your routes:

Endpoint	Limit	Window	On Exceed
POST /api/tenders/:id/bids/:id/submit	5 requests	10 seconds (per vendor_org_id)	HTTP 429
POST /api/auth/login	10 attempts	15 minutes (per IP)	HTTP 429 + 15-min lockout
All other API endpoints	200 requests	1 minute (per authenticated user)	HTTP 429

Task 28 — Audit Log Hardening & Final Review

The audit_logs table must be truly append-only. This final task locks it down and does a full review.

Database-level protection:

- Connect to PostgreSQL as the superuser (postgres). Run:

```
REVOKE UPDATE ON audit_logs FROM rfq_app_user;
REVOKE DELETE ON audit_logs FROM rfq_app_user;
```

- rfq_app_user is the database user your application connects with (from your .env). This prevents the app from ever modifying or deleting audit entries.

Final review checklist:

- Every controller that changes data has a corresponding INSERT INTO audit_logs.
- The action column uses consistent codes: TENDER_CREATED, TENDER_PUBLISHED, BID_SUBMITTED, ENVELOPE_OPENED, AWARD_ISSUED, etc.
- The metadata column captures context: for status changes include { old_status, new_status }; for bids include { vendor_org_id, version }.
- Run a quick test: perform a full tender lifecycle (create → publish → bid → evaluate → award). Then query audit_logs and confirm every step is recorded.

10. Quick Reference — All 28 Tasks at a Glance

#	Task	Phase	Key File(s) Created or Modified
1	Create folder structure	1	— (folders only)
2	Initialize backend + install packages	1	backend/package.json
3	Create .env config	1	backend/.env
4	Set up PostgreSQL database	1	— (run schema SQL in pgAdmin)
5	Database connection module	1	backend/src/config/database.js
6	Main server entry point	1	backend/src/app.js
7	User registration & login	1	controllers/authController.js, routes/authRoutes.js, middleware/authMiddleware.js
8	Tender CRUD API	2	controllers/tenderController.js, routes/tenderRoutes.js
9	Tender status transitions	2	services/tenderService.js
10	Line-item (BoM) API	2	controllers/itemController.js, routes/itemRoutes.js
11	Feature definitions & options API	2	controllers/featureController.js, routes/featureRoutes.js
12	Publish flow & vendor visibility	2	tenderController.js (updated), tenderService.js (updated)
13	Vendor profile & enlistment	3	controllers/vendorController.js, routes/vendorRoutes.js
14	Bid submission API	3	controllers/bidController.js, routes/bidRoutes.js
15	Bid versioning logic	3	services/bidService.js
16	Bid opening & envelope system	4	bidController.js (updated), services/envelopeService.js
17	Evaluation & scoring API	4	controllers/evalController.js, routes/evalRoutes.js
18	Comparison matrix API	4	controllers/comparisonController.js
19	Award API	4	controllers/awardController.js, routes/awardRoutes.js
20	Notification service (sender)	5	services/notificationService.js
21	Wire up all notification triggers	5	— (update multiple controllers)
22	Clarification (Q&A) API	5	controllers/clarificationController.js, routes/clarificationRoutes.js
23	Addenda API	5	controllers/addendumController.js, routes/addendumRoutes.js

24	Tax engine	6	controllers/taxController.js, services/taxService.js
25	Currency rate cache	6	services/fxService.js
26	Export engine (PDF & XLSX)	6	services/exportService.js, routes/exportRoutes.js
27	Rate limiting	6	app.js (updated)
28	Audit log hardening & final review	6	— (DB commands + review)